arXiv:2406.04520v1 [cs.CL] 6 Jun 2024

NATURAL PLAN: Benchmarking LLMs on Natural Language Planning

Huaixiu Steven Zheng **Swaroop Mishra** **Hugh Zhang***
Xinyun Chen **Minmin Chen** **Azade Nova** **Le Hou**
Heng-Tze Cheng **Quoc V. Le** **Ed H. Chi** **Denny Zhou**
Google DeepMind

Abstract

We introduce NATURAL PLAN, a realistic planning benchmark in natural language containing 3 key tasks: *Trip Planning*, *Meeting Planning*, and *Calendar Scheduling*. We focus our evaluation on the planning capabilities of LLMs with full information on the task, by providing outputs from tools such as Google Flights, Google Maps, and Google Calendar as contexts to the models. This eliminates the need for a tool-use environment for evaluating LLMs on Planning. We observe that NATURAL PLAN is a challenging benchmark for state of the art models. For example, in *Trip Planning*, GPT-4 and Gemini 1.5 Pro could only achieve 31.1% and 34.8% solve rate respectively. We find that model performance drops drastically as the complexity of the problem increases: all models perform below 5% when there are 10 cities, highlighting a significant gap in planning in natural language for SoTA LLMs. We also conduct extensive ablation studies on NATURAL PLAN to further shed light on the (in)effectiveness of approaches such as self-correction, few-shot generalization, and in-context planning with long-contexts on improving LLM planning.

1 Introduction

Large language models (LLMs) such as Gemini [Team et al., 2023] and GPT4 [Achiam et al., 2023] have demonstrated great potentials in solving complex tasks, such as understanding natural language [Winograd, 1972], reasoning [Cobbe et al., 2021] and code generation [Chen et al., 2021]. The planning capability of these models however are still far behind human performance [Xie et al., 2024]. Even though many recent works target to tackle this problem, without realistic benchmarks that reflect real-world applications, it is difficult to make meaningful progress in enhancing these models' planning abilities.

Planning [Russell and Norvig, 2016] has been exhaustively studied in robotics and embodied environments [LaValle, 2006, Camacho et al., 2007, Jiang et al., 2019a]. Automated planning algorithms have been widely used [Zhang et al., 2015, Lagriffoul et al., 2018, Jiang et al., 2019b, Ding et al., 2020] to enable planning in such systems. In these settings, planning involves coming up with a sequence of actions which when executed would take an agent from a certain initial state to a desired world state. These systems frequently use planning domain description language (PDDL) [McDermott et al., 1998] or answer set programming (ASP) [Brewka et al., 2011] as the underlying action language for the planners. Even though recent works have explored the potential of LLM-powered agents in classical planning settings [Silver et al., 2023, Liu et al., 2023, Hao et al., 2023, Valmeekam et al., 2024, Wang et al., 2023, Guan et al., 2024], planning scenarios are not in practical setting and they are not in natural language as they often contains templates.

*Currently at Scale AI, work done while interning at Google Deepmind

Preprint.

Trip Planning

You plan to visit 3 European cities for 14 days in total. You only take direct flights to commute between cities. You plan to stay in London for 4 days. You plan to stay in Bucharest for 5 days. You want to meet a friend in Bucharest between day 10 and day 14. You would like to visit Reykjavik for 7 days.

Here are the cities that have direct flights: London and Bucharest, Reykjavik and London .

Find a trip plan of visiting the cities for 14 days by taking direct flights to commute between them.

Figure 1: Illustrative example of the *Trip Planning* task.

Meeting Planning

You are visiting San Francisco for the day and want to meet as many friends as possible. Solve the problem by considering various different schedules and picking the best one to optimize your goals.

Travel distances (in minutes):
Marina District to Alamo Square: 15.
Alamo Square to Marina District: 15.

You arrive at Marina District at 9:00AM. James will be at Alamo Square from 3:45PM to 8:00PM. You'd like to meet James for a minimum of 75 minutes.

Figure 2: Illustrative example of the *Meeting Planning* task.

Calendar Scheduling

You need to schedule a meeting for Gerald and Melissa for half an hour between the work hours of 9:00 to 17:00 on Monday.

Here are the existing schedules for everyone during the day: Gerald is busy on Monday during 10:30 to 11:00, 12:00 to 14:00, 14:30 to 15:00; Melissa has meetings on Monday during 9:00 to 9:30, 10:00 to 11:00, 12:30 to 16:00;

The group would like to meet at their earliest availability. Find a time that works for everyone's schedule and constraints.

Figure 3: Illustrative example of the *Calendar Scheduling* task.

In this paper we introduce NATURAL PLAN, a new benchmark specifically designed to assess how well LLMs can handle planning tasks described in natural language. NATURAL PLAN includes three different planning tasks i.e., *Trip Planning*, *Meeting Planning*, *Calendar Scheduling*. For each task we gather real information from the existing tools. For example, in *Trip Planning* we query Google Flights API and provide the flight connectivity information between cities in the context. Figures 1- 3 illustrate the tasks along with associated google tools that provide information as context.

Our evaluation shows that NATURAL PLAN is a hard benchmark for state of the art models. For example, in the *Trip Planning task*, GPT-4 and Gemini 1.5 Pro achieves 31.1% and 34.8% performance, while GPT-3.5 is only at 7.3% and GPT-4o is at 3.7%. We find that model performance reduces significantly as the complexity of the task increases e.g. an increase in number of people, cities or days in the question. Self-correction does not help models in improving performance. We also observe interesting insights in our easy to hard and hard to easy generalization experiments. Lastly, we conduct study of in-context planning with long context, and show that Gemini 1.5 Pro significantly outperforms other models with $\sim 40\%$ accuracy on Trip Planning and $\sim 50\%$ on Calendar Scheduling.

2 NATURAL PLAN

NATURAL PLAN is a benchmark designed to evaluate capability of LLMs in performing planning tasks expressed in natural language. In this section, we discuss about various categories we have in NATURAL PLAN, our dataset construction process, quality measures and some statistics.

2.1 Dataset Categories

NATURAL PLAN has 3 planning categories: *Trip Planning*, *Meeting Planning* and *Calendar Scheduling*. Each of these is intended to capture a real world planning task involving some tools such as Google flights. We decouple tool-use from the reasoning task and provide tool outputs in-context to keep focus of NATURAL PLAN on planning. We believe this will significantly simplify the process of evaluating planning for LLMs.

2.1.1 *Trip Planning*

Trip Planning is a task focusing on planning a trip itinerary under given constraints. An example prompt and solution is shown in Appendix A. The task is to find the itinerary regarding the order of visiting N cities. We add enough constraints such that there is only one solution to the task, which makes the evaluation of the predictions straightforward.

2.1.2 *Meeting Planning*

This portion of the benchmark focuses on scheduling meetings under various given constraints. Given N friends, this task provides the meeting times as well as their locations. Appendix A demonstrates an example question. The objective is to meet as many friends as possible given the constraints, which include travel time between locations.

2.1.3 *Calendar Scheduling*

Calendar Scheduling is a task focusing on scheduling work meetings between multiple people given existing schedules and various constraints. We vary the number of people involved in the meeting and the number of workdays, in order to vary the task difficulties. Appendix A demonstrates an example question.

2.2 Dataset Construction

We create NATURAL PLAN synthetically by using corresponding tool data and creating various constraints. We describe below the specific creation process for each category of task.

2.2.1 *Trip Planning*

We create the dataset in the following way: To create a question, we randomly sample $N \in [3, 10]$ cities from the top-48 visited cities in Europe. We also randomly sample the total duration and the number of days spent in each city with $D \in [2, 7]$. We add constraints to the itinerary requiring visiting some cities during particular days, e.g. *meeting a friend in Paris from day 3 to day 5 on the trip*. As a global constraint, we add the preference of commuting between cities using direct flights only, excluding the possibility of visiting city Y after X if there is no direct flight from X to Y. To assist the LLM with the connectivity constraint, we query Google Flights API and provide the flight connectivity information between cities in the context.

2.2.2 *Meeting Planning*

For this dataset category, we randomly sample the person names to meet with and corresponding locations. We also sample time frames showing availability of persons. We add constraints in terms of availability of the subject e.g. you are visiting San Francisco for a day. The goal in this set up is to meet as many friends as possible. In order to make the problem realistic, the time constraints / locations follow real world locations. Travel distances are estimated by querying driving times via the Google Maps API.

Table 1: Statistics of NATURAL PLAN tasks.

Task	#Test Examples
Trip Planning	1600
Meeting Planning	1000
Calendar Scheduling	1000

2.2.3 *Calendar Scheduling*

This portion of the dataset includes tasks to schedule a meeting of either 30 minutes or an hour among various number of attendees in one or multiple of the work days. We instantiated two subsets: the first set fixes the meeting on a specific day, e.g., Monday while varying the number of attendees in $[2, 7]$; the second set fixes the number of attendees to 2 while varying the number of work days the meeting can be scheduled over in $[1, 5]$. Each person involved in the meeting comes with their existing meeting schedule during the day (first set) or the week (second set), which was created by adding random meetings of 30 minutes to the days until n hours of meeting time is filled. We randomly assign n to each attendee to make sure half of the attendees have busy schedule, meaning more than half of their working hours are spent in meetings, while the other half have light schedule with less than half of the working hours spent in meetings. Consecutive meetings of each attendee are combined into a single block as shown in Figure 3. Once we have the existing schedules of all attendees, we add two types of constraints: 1) earliest availability; 2) avoid meeting during certain time frame to make sure there is a single solution to the problem.

2.3 Statistics

Table 1 shows statistics of various tasks across *Trip Planning*, *Meeting Planning*, and *Calendar Scheduling* in NATURAL PLAN.

3 Experiment

We conduct various experiments to evaluate state-of-the-art LLMs on the planning tasks in NATURAL PLAN.

3.1 Models

We experiment several state-of-the-art LLMs: GPT-4 (gpt-4-turbo-2024-04-09) [Achiam et al., 2023], GPT-3.5 (gpt-3.5-turbo-0125), GPT-4o (gpt-4o-2024-05-13), and Gemini 1.5 Flash and Pro [Gemini Team, 2024].

3.2 Setups

Few-Shot
For each task in NATURAL PLAN, we present 5 examples within the same task as few-shot exemplars to enable the LLM in-context learning [Brown et al., 2020] from the examples (see Appendix B for the few-shot prompt). This setup also ensures that outputs can be easily parsed for scoring.

Constraint Complexity Understanding
To gain a better understanding of the planning ability of LLMs, we vary the difficulty level of each task through a controlled variable. We conduct analysis by varying constraints such as number of people, number of cities, number of days etc. to better understand behavior of models across the variation in the complexity of the planning task.

Few-Shot Generalization
We conduct easy-to-hard and hard-to-easy generalization study by providing easier and harder examples as in-context exemplars, respectively. For instance, for easy-to-hard (hard-to-easy) generalization

in *Trip Planning*, we provide 5 easier (harder) examples with 2 fewer (more) cities than the ones in the evaluation task instances.

Self-Correction
We evaluate if models can self-correct their mistakes across various categories of tasks in NATURAL PLAN. Here we prompt the model to identify if there are any mistakes and self correct if necessary.

In-Context Planning with Long Context
We evaluate how well models can leverage the long-context capabilities for in-context learning in planning. We split the test-set into 80% and 20%, where 80% is used as in-context examples while we evaluate the model performance on 20% of the test-set. We increase the number of few-shot examples used up to 800.

Evaluation
We parse the output from the LLM, and compare it with the golden plan. We compute an exact match (EM) score of the LLM-generated plan being identical to the golden plan. See Appendix C for the details of parsing and scoring.

4 Results and Analysis

Table 2: Accuracy of 5 models on NATURAL PLAN.

Task	GPT-3.5	GPT-4	GPT-4o	Gemini 1.5 Flash	Gemini 1.5 Pro
Trip Planning	7.3%	31.1%	3.7%	25.6%	**34.8%**
Meeting Planning	19.1%	**47.0%**	45.2%	23.9%	39.1%
Calendar Scheduling	19.9%	41.2%	43.7%	34.3%	**48.9%**

4.1 Results

Table 2 shows the performance of various LLMs across tasks in NATURAL PLAN in the 5-shot setup. We get the following insights from the results.

All models perform poorly on NATURAL PLAN. Gemini 1.5 Pro performs the best on *Trip Planning* and *Calendar Scheduling* while GPT-4 does the best on *Meeting Planning*.

In *Trip Planning*, Gemini 1.5 Pro achieves 34.8% accuracy while GPT-4 is 31.1%. In particular, we observe that GPT-4o is only at 3.7%. Further analysis shows that GPT-4o is struggling to understand and respect the flight connectivity and travel date constraints, see Appendix D for a more detailed analysis. GPT-4 and GPT-4o achieve 47% and 45.2% accuracy on *Meeting Planning* while Gemini 1.5 Pro is 39.1%. For *Calendar Scheduling*, Gemini 1.5 Pro outperforms GPT-4 and GPT-4o at 48.9%.

Comparing the three tasks in NATURAL PLAN, *Trip Planning* is the hardest among them with the lowest best accuracy of 34.8%. Note all models have accuracy below 50% on all three tasks in NATURAL PLAN, highlighting that planning in natural language is still a very challenging task even for the state of the art models, despite that all tool-use information needed is provided in the context.

4.2 Constraint Complexity

Figures 4- 6 show the performance variation of models across variation in number of cities in *Trip Planning*, number of people in *Meeting Scheduling*, and number of attendees/days in *Calendar Scheduling* . We observe the following key insights:

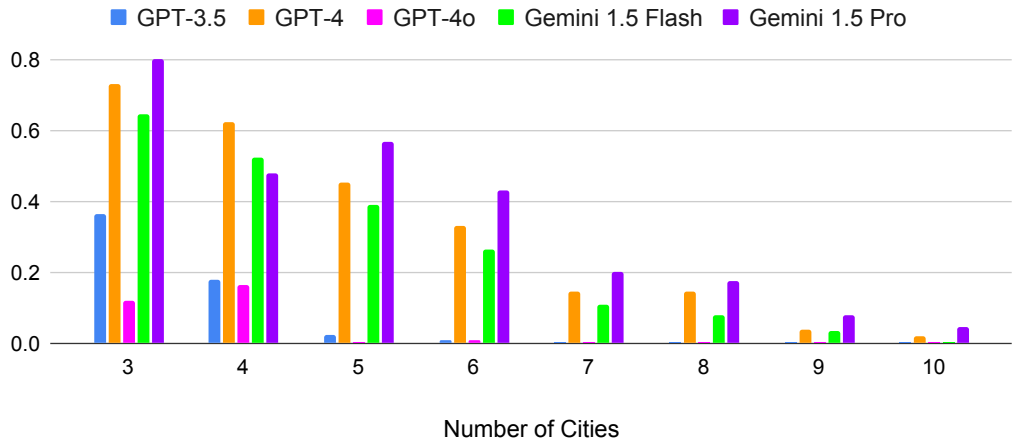

Figure 4: Constraint complexity analysis of the *Trip Planning* task in NATURAL PLAN as a function of number of cities.

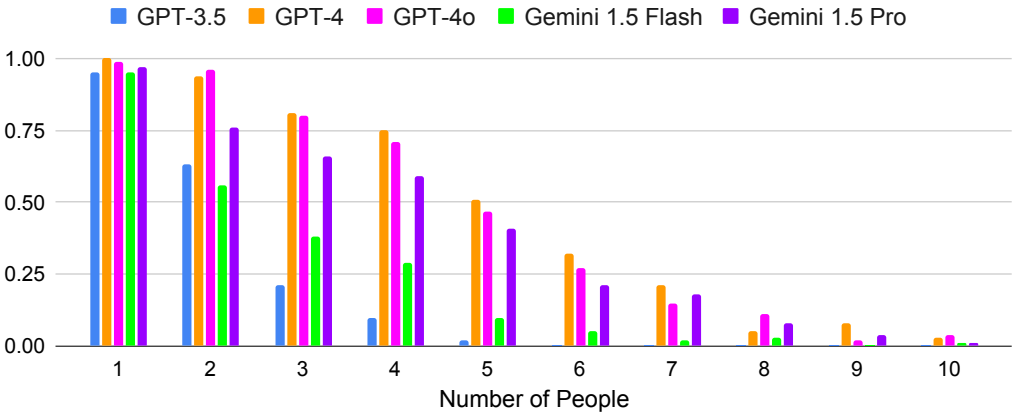

Figure 5: Constraint complexity analysis of the *Meeting Planning* task in NATURAL PLAN as a function of number of people.

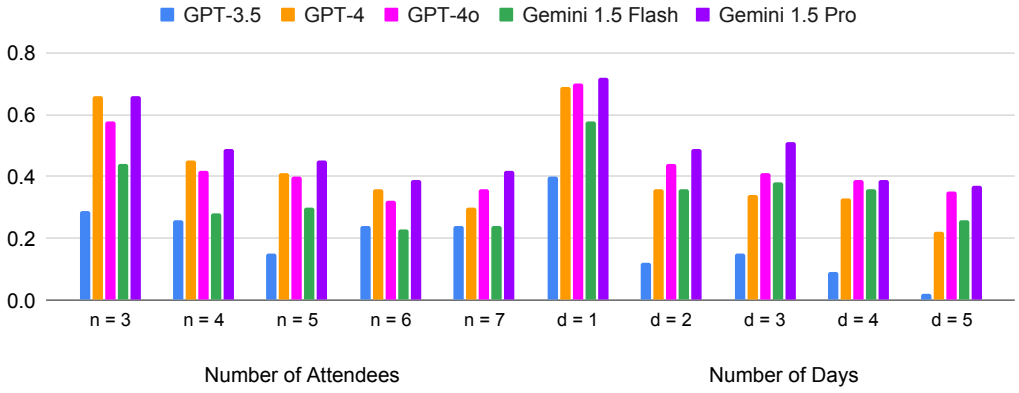

Figure 6: Constraint complexity analysis of the *Calendar Scheduling* task as a function of the number of people involved to meet during a single day (left), and number of working days to choose a meeting time from between two attendees (right).

Model performance reduces significantly as the task complexity increases as number of cities/number of people/number of attendees/number of days increase. For *Trip Planning*, Figure 4 shows that models start to fail with more cities involved in the planning. In *Trip Planning*, all five models perform below 5% when there are 10 cities. For *Meeting Planning*, we evaluate scenarios with the number of people to meet from 1 to 10. Figure 5 again shows that the performance drops sharply with more people to meet. Beyond 8 people, all models have a score lower than 10%. For *Calendar Scheduling*, the drop of accuracy is not as significant when increasing number of attendees or number of days compared to the other two tasks.

4.3 Generalization Analysis

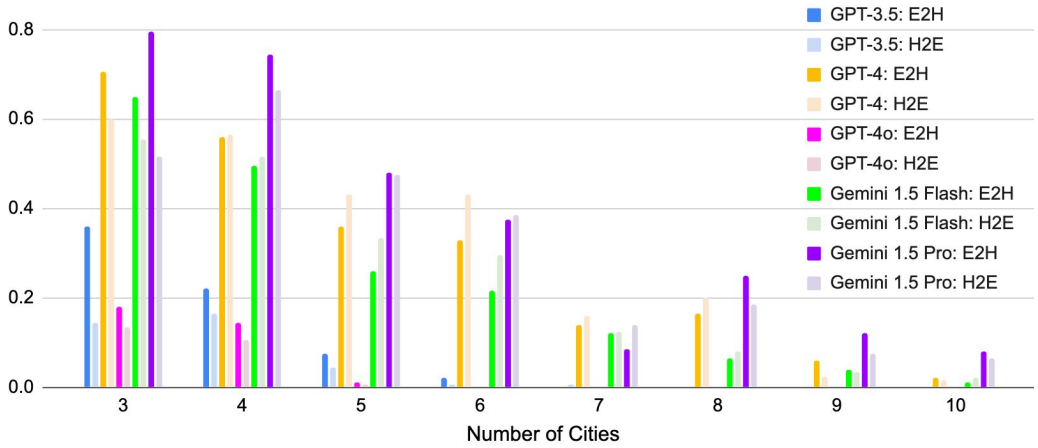

Figure 7: Model performance on the *Trip Planning* task as a function of number of cities for few-shot generalization ablations: E2H=easy-to-hard, H2E=hard-to-easy.

Figure 7 shows the summary of our easy-to-hard and hard-to-easy generalization experiments. For easy-to-hard (hard-to-easy) generalization, we present 5 easier (harder) examples with 2 fewer (more) cities than the task, to test whether LLMs can generalize from easier (harder) in-context learning examples. We choose *Trip Planning* to conduct our ablations on given that it is the most difficult task among NATURAL PLAN tasks.

In general, we observe that easy-to-hard performs better than hard-to-easy generalization across model families. Section 4.2 shows models perform worse when the task complexity is higher. This result indicates that understanding and making use of hard demonstrations in in-context learning is still challenging even for state of the art LLMs. However, we do observe that for GPT-4 and Gemini 1.5 Flash, the trend is reversed as the task complexity increases: for $N = 5 \sim 8$, hard-to-easy demonstrations work better than easy-to-hard.

4.4 Self-Correction

Figure 8: Accuracy of *Trip Planning* as a function of number of cities for self-correction ablations. SC=self-correction

Figure 8 shows the results of performing self-correction on model predictions in the *Trip Planning* task. We prompt the LLM to check whether the generated plan is correct or not, and if determined incorrect, fix the plan (see Appendix B for the prompt).

Self-correction leads to significant model performance drop across all models. Interestingly, the stronger models such as GPT-4 and Gemini 1.5 Pro suffer bigger loss compared to GPT-3.5. This aligns well with previous findings that self-correction could lead to performance degradation on Reasoning [Huang et al., 2023]. The stronger instruction-tuned models like GPT-4 and Gemini 1.5 Pro could be over-confident towards correcting its own solutions when prompted, see Appendix E for examples of self-correction.

4.5 In-Context Planning with Long Context

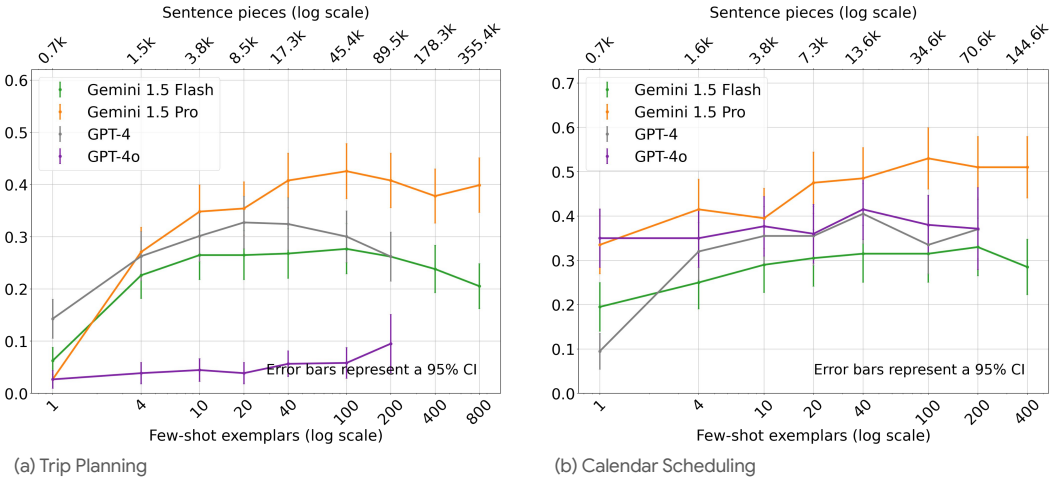

Figure 9: In-Context planning with long context for *Trip Planning* and *Calendar Scheduling* tasks. Gemini 1.5 Pro shows strong long-context learning on Planning compared to other models.

In this section, we study the possibility of in-context planning by leveraging the long context capabilities of recent models [Gemini Team, 2024]. Figure 9 shows the performance of four models up to 800 shots.

Due to the strong long context capabilities, Gemini Pro 1.5 is able to leverage more in-context examples up to 355K tokens, still showing steady improvements. For instance, in *Trip Planning*, increasing number of shots from 1 to 800 brings up the accuracy of Gemini Pro 1.5 from 2.7% to 39.9%. In the mean time, both GPT-4 and Gemini 1.5 Flash peak at 20-shot and start to show performance degradation. The same is observed in *Calendar Scheduling* where Gemini 1.5 Pro continually improves up to 400 shots. These results show the promise of in-context planning where the long-context capabilities enable LLMs to leverage further context to improve Planning.

5 Related Works

Planning empowers intelligent agents to look ahead and proactively determine a course of action to reach objectives [McCarthy et al., 1963, Russell and Norvig, 2016]. Automated/classical planning is a subarea of AI that allows sequential decision-making systems to choose a sequence of actions using which they complete a (set of) task(s) autonomously [Ghallab et al., 2004]. Most of the planning formalisms use a Planning Domain Definition Language (PDDL) [McDermott et al., 1998], where it separates the environment dynamics (domain definition) from the environment configuration (problem definition). Domain-independent planners like Fast-Forward [Hoffmann and Nebel, 2001], Fast-Downward [Helmert, 2006], etc. take these domain and problem definitions as input and generate a sequence of actions that takes a planning agent from a given initial state to a state satisfying a goal condition (also given as input with the problem definition). Such classical planning methods have been widely adopted in robots and embodied environments [Camacho et al., 2007, Jiang et al., 2019a]. Most planning benchmarks [2] use similar formulations with variations like probabilities [Younes et al., 2005], numeric fluents [3], temporal actions, etc. as benchmarks. The main issue with such benchmarks is that they are not expressed in natural language and hence need experts to handcraft the domains and problems.

Recently prompting LLMs to extract common sense knowledge gained attention [Huang et al., 2022, Singh et al., 2023, Ding et al., 2023]. Effectiveness of LLMs in generating plans has been studied by Valmeekam et al. [2024], Hao et al. [2023], Guan et al. [2024]. While Valmeekam et al. [2024], Guan et al. [2024] translate natural language instructions into the executable programming languages, such as PDDL, and runs classical planning algorithms, Hao et al. [2023] uses the LLM as both a world model and a reasoning agent, enhanced by a Monte Carlo Tree Search algorithm for efficient exploration.

Even though these works target to improve the planning capability of LLMs, these are not able to perform well in more complex and realistic settings, closer to those in which humans operate [Valmeekam et al., 2022]. TravelPlanner [Xie et al., 2024] provides a sandbox with various tools to evaluate language model capabilities in performing planning via agents. Our work NATURAL PLAN provides a holistic benchmark in natural language to evaluate the planning capability of models in real world tasks.

6 Conclusion

We introduce NATURAL PLAN to evaluate the capability of LLMs to plan on real world tasks given in natural language. NATURAL PLAN covers three domains: *Trip Planning*, *Calendar Scheduling*, and *Meeting Planning*. The tasks also contain API results from various tools such as Google Calendar, Google Flights and Google Maps mimicking real world planning tasks with tools. We find that NATURAL PLAN is very hard for state of the art models to solve. Models increasingly struggle as the complexity of the problem increases, e.g. an increase in number of people, cities or days in the question. However, our in-context planning experiments demonstrate the potential promise of leveraging the long-context capabilities to boost LLMs' planning ability.

Acknowledgments

We acknowledge careful review and feedback from Uri Alon and Aleksandra Faust of Google DeepMind on the draft of this paper.

[2] https://www.icaps-conference.org/competitions/

[3] https://ipc2023-numeric.github.io/

References

Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

Gerhard Brewka, Thomas Eiter, and Mirosław Truszczyński. Answer set programming at a glance. *Communications of the ACM*, 54(12):92–103, 2011.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

Eduardo F Camacho, Carlos Bordons, Eduardo F Camacho, and Carlos Bordons. *Constrained model predictive control*. Springer, 2007.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

Yan Ding, Xiaohan Zhang, Xingyue Zhan, and Shiqi Zhang. Task-motion planning for safe and efficient urban driving. In *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 2119–2125. IEEE, 2020.

Yan Ding, Xiaohan Zhang, Chris Paxton, and Shiqi Zhang. Task and motion planning with large language models for object rearrangement. In *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 2086–2092. IEEE, 2023.

Google Gemini Team. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context, 2024.

Malik Ghallab, Dana Nau, and Paolo Traverso. *Automated Planning: Theory and Practice*. Morgan Kaufmann, Amsterdam, 2004. ISBN 978-1-55860-856-6. URL `http://www.sciencedirect.com/science/book/9781558608566`.

Lin Guan, Karthik Valmeekam, Sarath Sreedharan, and Subbarao Kambhampati. Leveraging pre-trained large language models to construct and utilize world models for model-based task planning. *Advances in Neural Information Processing Systems*, 36, 2024.

Shibo Hao, Yi Gu, Haodi Ma, Joshua Jiahua Hong, Zhen Wang, Daisy Zhe Wang, and Zhiting Hu. Reasoning with language model is planning with world model. *arXiv preprint arXiv:2305.14992*, 2023.

Malte Helmert. The fast downward planning system. *Journal of Artificial Intelligence Research*, 26: 191–246, 2006.

Jörg Hoffmann and Bernhard Nebel. The FF Planning System: Fast Plan Generation Through Heuristic Search. *Journal of Artificial Intelligence Research*, 14:253–302, 2001.

Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. In *The Twelfth International Conference on Learning Representations*, 2023.

Wenlong Huang, Fei Xia, Ted Xiao, Harris Chan, Jacky Liang, Pete Florence, Andy Zeng, Jonathan Tompson, Igor Mordatch, Yevgen Chebotar, et al. Inner monologue: Embodied reasoning through planning with language models. *arXiv preprint arXiv:2207.05608*, 2022.

Yu-qian Jiang, Shi-qi Zhang, Piyush Khandelwal, and Peter Stone. Task planning in robotics: an empirical comparison of pddl-and asp-based systems. *Frontiers of Information Technology & Electronic Engineering*, 20:363–373, 2019a.

Yuqian Jiang, Harel Yedidsion, Shiqi Zhang, Guni Sharon, and Peter Stone. Multi-robot planning with conflicts and synergies. *Autonomous Robots*, 43(8):2011–2032, 2019b.

Fabien Lagriffoul, Neil T Dantam, Caelan Garrett, Aliakbar Akbari, Siddharth Srivastava, and Lydia E Kavraki. Platform-independent benchmarks for task and motion planning. *IEEE Robotics and Automation Letters*, 3(4):3765–3772, 2018.

Steven M. LaValle. *Planning Algorithms*. Cambridge University Press, 2006.

Bo Liu, Yuqian Jiang, Xiaohan Zhang, Qiang Liu, Shiqi Zhang, Joydeep Biswas, and Peter Stone. Llm+ p: Empowering large language models with optimal planning proficiency. *arXiv preprint arXiv:2304.11477*, 2023.

John McCarthy et al. *Situations, actions, and causal laws*. Comtex Scientific, 1963.

Drew McDermott, Malik Ghallab, Adele Howe, Craig Knoblock, A. Ram, Manuela Veloso, Daniel S. Weld, and David Wilkins. PDDL – The Planning Domain Definition Language. Technical Report CVC TR-98-003/DCS TR-1165, Yale Center for Comp. Vision and Control, 1998.

Stuart J Russell and Peter Norvig. *Artificial intelligence: a modern approach*. Pearson, 2016.

Tom Silver, Soham Dan, Kavitha Srinivas, Joshua B Tenenbaum, Leslie Pack Kaelbling, and Michael Katz. Generalized planning in pddl domains with pretrained large language models. *arXiv preprint arXiv:2305.11014*, 2023.

Ishika Singh, Valts Blukis, Arsalan Mousavian, Ankit Goyal, Danfei Xu, Jonathan Tremblay, Dieter Fox, Jesse Thomason, and Animesh Garg. Progprompt: Generating situated robot task plans using large language models. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 11523–11530. IEEE, 2023.

Gemini Team, Rohan Anil, Sebastian Borgeaud, Yonghui Wu, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, et al. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023.

Karthik Valmeekam, Alberto Olmo, Sarath Sreedharan, and Subbarao Kambhampati. Large language models still can't plan (a benchmark for llms on planning and reasoning about change). *arXiv preprint arXiv:2206.10498*, 2022.

Karthik Valmeekam, Matthew Marquez, Sarath Sreedharan, and Subbarao Kambhampati. On the planning abilities of large language models-a critical investigation. *Advances in Neural Information Processing Systems*, 36, 2024.

Xinyuan Wang, Chenxi Li, Zhen Wang, Fan Bai, Haotian Luo, Jiayou Zhang, Nebojsa Jojic, Eric P Xing, and Zhiting Hu. Promptagent: Strategic planning with language models enables expert-level prompt optimization. *arXiv preprint arXiv:2310.16427*, 2023.

Terry Winograd. Understanding natural language. *Cognitive psychology*, 3(1):1–191, 1972.

Jian Xie, Kai Zhang, Jiangjie Chen, Tinghui Zhu, Renze Lou, Yuandong Tian, Yanghua Xiao, and Yu Su. Travelplanner: A benchmark for real-world planning with language agents. *arXiv preprint arXiv:2402.01622*, 2024.

Håkan LS Younes, Michael L Littman, David Weissman, and John Asmuth. The first probabilistic track of the international planning competition. *Journal of Artificial Intelligence Research*, 24: 851–887, 2005.

Shiqi Zhang, Fangkai Yang, Piyush Khandelwal, and Peter Stone. Mobile robot planning using action language with an abstraction hierarchy. In *International Conference on Logic Programming and Nonmonotonic Reasoning*, pages 502–516. Springer, 2015.

A NATURAL PLAN Examples

Below is one example pair of the prompt and solution for each task in NATURAL PLAN.

Trip Planning

Prompt:
You plan to visit 3 European cities for 14 days in total. You only take direct flights to commute between cities. You would like to visit Florence for 6 days. You want to meet a friend in Florence between day 9 and day 14. You would like to visit Barcelona for 5 days. You would like to visit Helsinki for 5 days.

Here are the cities that have direct flights: Barcelona and Florence, Helsinki and Barcelona.

Find a trip plan of visiting the cities for 14 days by taking direct flights to commute between them.

Solution:
Here is the trip plan for visiting the 3 European cities for 14 days:
Day 1-5: Arriving in Helsinki and visit Helsinki for 5 days.
Day 5: Fly from Helsinki to Barcelona.
Day 5-9: Visit Barcelona for 5 days.
Day 9: Fly from Barcelona to Florence.
Day 9-14: Visit Florence for 6 days.

Meeting Planning

Prompt:
You arrive at SOMA (South of Market) at 9:00AM. Joseph will be at Alamo Square from 4:15PM to 6:30PM. You'd like to meet Joseph for a minimum of 105 minutes. Andrew will be at Haight-Ashbury from 4:30PM to 6:15PM. You'd like to meet Andrew for a minimum of 30 minutes. John will be at Nob Hill from 7:15AM to 11:00AM. You'd like to meet John for a minimum of 75 minutes.

It takes 27 minutes to travel between Sunset District and Nob Hill via car. It takes 18 minutes to travel between Sunset District and Alamo Square via car. It takes 11 minutes to travel between Sunset District and Golden Gate Park via car. It takes 25 minutes to travel between Nob Hill and Sunset District via car. It takes 11 minutes to travel between Nob Hill and Alamo Square via car. It takes 18 minutes to travel between Nob Hill and Golden Gate Park via car. It takes 16 minutes to travel between Alamo Square and Sunset District via car. It takes 11 minutes to travel between Alamo Square and Nob Hill via car. It takes 9 minutes to travel between Alamo Square and Golden Gate Park via car. It takes 10 minutes to travel between Golden Gate Park and Sunset District via car. It takes 19 minutes to travel between Golden Gate Park and Nob Hill via car. It takes 10 minutes to travel between Golden Gate Park and Alamo Square via car.

Solution:
You start at SOMA (South of Market) at 9:00AM. You travel to Nob Hill in 10 minutes and arrive at 9:10AM. You meet John for 75 minutes from 9:10AM to 10:25AM. You travel to Alamo Square in 11 minutes and arrive at 10:36AM. You wait until 4:15PM. You meet Joseph for 105 minutes from 4:15PM to 6:00PM.

Calendar Scheduling

Prompt:
You need to schedule a meeting for Harold and Patrick for half an hour between the work hours of 9:00 to 17:00 on Monday.

Harold's calendar is wide open the entire day. Patrick is busy on Monday during 9:00 to 9:30, 10:30 to 12:00, 12:30 to 13:30, 14:00 to 14:30, 15:00 to 16:30;

Find a time that works for everyone's schedule and constraints.

Solution:
Here is the proposed time: Monday, 9:30 - 10:00

B Prompts

Here is the 5-shot prompt for inference of *Trip Planning*. Similar prompts are used for *Meeting Planning*, and *Calendar Scheduling*.

Standard 5-Shot Prompt for *Trip Planning*

Prompt:
You are an expert at planning trips. You are given a few constraints regarding the cities to visit and the durations of staying at each city. You are also given the flight information between the cities. Here are a few example tasks and solutions:
TASK: <task1>

SOLUTION: <solution1>
...
TASK: <task5>

SOLUTION: <solution5>

TASK: <task>

SOLUTION:

Here is the prompt used for easy-to-hard and hard-to-easy generalization ablations of *Trip Planning*:

Easy-to-Hard/Hard-to-Easy Prompt for *Trip Planning*

Prompt:
You are an expert at planning trips. You are given a few constraints regarding the cities to visit and the durations of staying at each city. You are also given the flight information between the cities. Here are a few example tasks and solutions:
TASK: <easy/hard task1>

SOLUTION: <solution1>
...
TASK: <easy/hard task5>

SOLUTION: <solution5>

TASK: <task>

SOLUTION:

Here is the self-correction prompt used for *Trip Planning* self-correction ablation:

Self-Correction Prompt for *Trip Planning*

Prompt:
You are an expert at planning trips. You are given a TASK of Trip Planning request, and a proposed SOLUTION. Your job is to identify any error in the SOLUTION, correct the error and generate a new solution. If there is no error, simple output the proposed SOLUTION.

TASK: <task>

SOLUTION: <model output>

Please self correct the solution if there is any error and output your solution in the following format and keep the NEW SOLUTION in the same format as SOLUTION:
ERROR IDENTIFIED:
NEW SOLUTION:

C Plan Parsing and Scoring

In this section, we describe the details on how to parse the plans generated by models, and score the exact-match score against the ground-truth plans. We rely on the few-shot exemplars to make sure the generated plans from models follow the same template solutions provided in few-shot examples. Then, we rely on regex expressions to extract the exact date/place/time information to compare against the ground-truth plans. For example, in *Trip Planning*, we use a regex expression *r".*Day (\d+).*from (\w+) to (\w+)"* to parse the date D of flight from city X to Y. We do similar regex parsing to the other two tasks. For exact match score, only if all the information (e.g. D, X and Y) matches, we score the plan to be correct.

D Analysis of GPT-4o on *Trip Planning*

We sample 10 examples where GPT-4o made an error on *Trip Planning*, and annotate the type of errors GPT-4o made. All of the errors are due to the fact that GPT-4o failed to understand and respect the constraints in the task. 7/10 are failures in respecting the Flight Connectivity constraints while the other 3/10 are due to Travel Date constraint failures. Below shows an example for both cases.

GPT-4o Error on *Trip Planning*: Flight Connectivity

Task:
You plan to visit 3 European cities for 14 days in total. You only take direct flights to commute between cities. You would like to visit Florence for 6 days. You want to meet a friend in Florence between day 9 and day 14. You would like to visit Barcelona for 5 days. You would like to visit Helsinki for 5 days.

Here are the cities that have direct flights: Barcelona and Florence, Helsinki and Barcelona.

Find a trip plan of visiting the cities for 14 days by taking direct flights to commute between them.

Solution:
Here is the trip plan for visiting the 3 European cities for 14 days:

Day 1-5: Arriving in Helsinki and visit Helsinki for 5 days.
Day 5: Fly from Helsinki to Barcelona.
Day 5-9: Visit Barcelona for 5 days.
Day 9: Fly from Barcelona to Florence.
Day 9-14: Visit Florence for 6 days.

GPT-4o Output:
"Here is the trip plan for visiting the 3 European cities for 14 days:

Day 1-5: Arriving in Barcelona and visit Barcelona for 5 days.
Day 5: Fly from Barcelona to Helsinki.
Day 5-10: Visit Helsinki for 5 days.
Day 10: Fly from Helsinki to Florence.
Day 10-14: Visit Florence for 6 days.

GPT-4o Error on *Trip Planning*: Travel Date

Task:
You plan to visit 3 European cities for 8 days in total. You only take direct flights to commute between cities. You would like to visit Dubrovnik for 3 days. You would like to visit Berlin for 4 days. During day 1 and day 4, you have to attend a conference in Berlin. You would like to visit Munich for 3 days.

Here are the cities that have direct flights: Munich and Dubrovnik, Berlin and Munich.
Find a trip plan of visiting the cities for 8 days by taking direct flights to commute between them.

Solution:
"Here is the trip plan for visiting the 3 European cities for 8 days:

Day 1-4: Arriving in Berlin and visit Berlin for 4 days.
Day 4: Fly from Berlin to Munich.
Day 4-6: Visit Munich for 3 days.
Day 6: Fly from Munich to Dubrovnik.
Day 6-8: Visit Dubrovnik for 3 days."

GPT-4o Output:
"ere is the trip plan for visiting the 3 European cities for 8 days:
Day 1-4: Arriving in Berlin and attend the conference while visiting Berlin for 4 days.
Day 4: Fly from Berlin to Munich.
Day 4-7: Visit Munich for 3 days.
Day 7: Fly from Munich to Dubrovnik.
Day 7-10: Visit Dubrovnik for 3 days.

E Self-Correction Example

Here we show an example of self-correction on *Trip Planning*, to illustrate the model behavior after self-correction. The errors made by the models are highlighted in red.

Self-Correction Example

Task:
You plan to visit 3 European cities for 14 days in total. You only take direct flights to commute between cities. You would like to visit Florence for 6 days. You want to meet a friend in Florence between day 9 and day 14. You would like to visit Barcelona for 5 days. You would like to visit Helsinki for 5 days.

Here are the cities that have direct flights: Barcelona and Florence, Helsinki and Barcelona.

Find a trip plan of visiting the cities for 14 days by taking direct flights to commute between them.
Solution:
Here is the trip plan for visiting the 3 European cities for 14 days:
Day 1-5: Arriving in Helsinki and visit Helsinki for 5 days.
Day 5: Fly from Helsinki to Barcelona.
Day 5-9: Visit Barcelona for 5 days.
Day 9: Fly from Barcelona to Florence.
Day 9-14: Visit Florence for 6 days.

GPT-3.5 Output:
ERROR IDENTIFIED: The error in the solution is that the friend's visit in Florence is scheduled between day 10 and day 14, instead of between day 9 and day 14 as requested.
NEW SOLUTION: Here is the corrected trip plan for visiting the 3 European cities for 14 days:
Day 1-5: Arriving in Helsinki and visit Helsinki for 5 days.
Day 5: Fly from Helsinki to Barcelona.
Day 5-10: Visit Barcelona for 5 days.
Day 10: Fly from Barcelona to Florence.
Day 10-15: Visit Florence for 6 days.
GPT-4 Output:
ERROR IDENTIFIED: The proposed solution has an error in the total number of days spent. The total number of days spent in the proposed solution is 15 days (5 days in Helsinki, 5 days in Barcelona, and 6 days in Florence), which is more than the 14 days available for the trip.
NEW SOLUTION: Here is the corrected trip plan for visiting the 3 European cities for 14 days:
Day 1-4: Arriving in Helsinki and visit Helsinki for 4 days.
Day 5: Fly from Helsinki to Barcelona.
Day 5-9: Visit Barcelona for 5 days.
Day 10: Fly from Barcelona to Florence.
Day 10-14: Visit Florence for 5 days.
Gemini Pro 1.5 Output:
"**ERROR IDENTIFIED:** The proposed solution has overlapping days. For example, Day 5 is used for both visiting Helsinki and flying to Barcelona. Additionally, the solution doesn't account for the fact that there are no direct flights between Helsinki and Florence.

NEW SOLUTION: Here is the trip plan for visiting the 3 European cities for 14 days:
Day 1-5: Arriving in Barcelona and visit Barcelona for 5 days.
Day 6: Fly from Barcelona to Florence.
Day 6-11: Visit Florence for 6 days (meeting the friend during this time).
Day 12: Fly from Florence to Barcelona.
Day 12-13: Spend an extra day in Barcelona.
Day 14: Fly from Barcelona to Helsinki.